Компиляторы. Язык ассемблера RISC-V 64

Косарев Дмитрий

13 сентября 2025 г.

Дата сборки: 12 сентября 2025 г.



Пример: Hello, World!

```
.section .data # RW section
HELLO_WORLD: .asciz "Hello, World!"
.equ BUFSIZE, 13 # Another constant

.text
.global _start # Entry point
_start:
  li a7, 64  # system call 'write'
  li a0, 1 # stdout
  la a1, HELLO_WORLD
  li a2, BUFSIZE
  ecall
  li a7, 94 # system call 'exit_group'
  li a0, 0 # return 0
  ecall # perform syscall
```

li — **l**oad **i**mmediate
la — **l**oad **a**ddress
ecall — **e**nvironment **c**all

Секции .text и .data

rem — **rem**inder (остаток от деления)
ble — **b**ranch if **l**ess or **e**qual
ret — **ret**urn

Пример: факториал

```
.global _start
_start: li a0, 6 # cmd destination, source
        call fac # result is in a0. N.B. maybe BUG?
        li a5, 256
        rem a0, a0, a5 # 6! - 256*2 = 208
        li  a7, 93 # System call number 93 terminates
        ecall      # Ask Linux to perform system call
fac:  li   a1, 1  # 1
      ble  a0, a1, .fac_exit     # branch if less or equal
      mv   a5, a0 # a0 is input
      li   a0, 1 # product, return value
.fac_loop:    mv   a4, a5
              addi a5, a5, -1
              mul  a0, a4, a0
              bne  a5, a1, .fac_loop # branch if not equal
.fac_exit: ret
```

Как это (кросс-)компилировать и запускать?

```
$ sudo apt install --yes \
    binutils-riscv64-linux-gnu gdb-multiarch qemu-user
...
$ riscv64-linux-gnu-as -march=rv64gc input.S -o temp.o
$ riscv64-linux-gnu-ld temp.o -o file.exe
$ qemu-riscv64 -L /usr/riscv64-linux-gnu -cpu rv64 ./file.exe
$ echo $?
[208]
```

Варьировать можно выделенное цветом

Как вообще это работает?

Обычно:

- Запускается программа. Linux передает управление в `_start`
- Работает «инициализация» стандартной библиотеки языка C. Передается управление в main
- Работает функция main.
  `int main(...) ... return 0;`
  Возвращается код программы
- Результат main передается в Linux через системный вызов. Конец

У нас:

- Так же
- Инициализации в примере выше нет. (Для более сложных программ она нужна)
- Код _start и main свален в кучу
- Так же передаем результат в операционную систему

---

https://github.com/bminor/glibc/blob/master/sysdeps/riscv/start.S (проверено: 11 октября 2024 г.)

Как это дебажить через GDB? (1/3)

В одном терминале:

```
qemu-riscv64 -L /usr/riscv64-linux-gnu -cpu rv64 -g 1234
fac/rv64.exe
```

В другом:

```
(cd fac && gdb-multiarch rv64.exe)
```

Перед этим создать файл инициализации `test/.gdbinit` и разрешить его использование (при запуске GDB скажут как именно)

P.S. Варьировать можно порт `1234` и `fac/rv64.exe`

Как это дебажить через GDB? (2/3)

```
 0 $ cat tests/.gdbinit
 1 set history save
 2 set architecture riscv:rv64
 3 set sysroot /usr/riscv64-linux-gnu
 4 target remote :1234
 5 tui new-layout example {-horizontal regs 1 asm 1} 2
      status 0 cmd 1
 6 tui layout example
 7 # tui disable
 8 focus cmd
 9 b _start
10 c
11 # x/8xg $sp
```

Как это дебажить через GDB? (3/3)

Другие полезные сведения/команды:

- Вход внутрь инструкции (например, call) делается через `si` (step into)
- Перешагивание — `ni` (**n**ext **i**nstruction; step over)
- ENTER повторяет предыдущую команду
- Значения регистров и стека во время отладки можно печатать

```
(gdb) printf "%X␣%d\n", $a1, $a1
12000 73728
(gdb) x/8xg $sp
0x2aaaab2aa620: 0x0000000000000001      0x00002aaaab2aa9c1
0x2aaaab2aa630: 0x0000000000000000      0x00002aaaab2aa9ce
0x2aaaab2aa640: 0x00002aaaab2aa9dd      0x00002aaaab2aa9f1
0x2aaaab2aa650: 0x00002aaaab2aaa27      0x00002aaaab2aaa3f
```

Целочисленные регистры (источник: [4])

Регистр	Имя в ABI	Описание	Сохраняет
x0	zero	zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5–x7	t0–t2	Вре́менные	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Callee-saved registers	Callee
x10–x17	a0–a7	Аргументы функций	Caller
x18–x27	s2–s11	Callee-saved registers	Callee
x28–x31	t3–t6	Вре́менные	Caller

Calling conventions. История [2]. System V CC [3]
caller-saved (call globerred) vs callee-saved (call-preserved)

Выделение памяти на стеке (1/2)

```
(gdb) x/4xg $sp
0x2aaaab2aa620: 0x0000000000000001 0x00002aaaab2aa9c1
0x2aaaab2aa630: 0x0000000000000000 0x00002aaaab2aa9ce
(gdb) ni                                   addi sp, sp, -16
(gdb) ni                                   sd a0, 8(sp)
(gdb) x/4xg $sp
0x2aaaab2aa610: 0x0000000000000000 0x0000000000000078
0x2aaaab2aa620: 0x0000000000000001 0x00002aaaab2aa9c1
(gdb) ni                                   sd ra, (sp)
(gdb) x/4xg $sp
0x2aaaab2aa610: 0x000000000001023a 0x0000000000000078
0x2aaaab2aa620: 0x0000000000000001 0x00002aaaab2aa9c1
```

Выделение памяти на стеке (2/2)

```
.global print_int
print_int:              # Function
    addi sp, sp, -16
    sd a0, 8(sp)
    sd ra, (sp)         # IMPORTANT
    la a0, str2
    li a1, 0x20 # space
    li a2, BUFSIZE
    call memset         # Function call
    ...
```

Грабли:

- Забыть сохранить ra
- Перепутать sd (store double word, 64bit) и sw (store word, 32 bit)

The RISC-V Instruction Set Manual Volume I: User-Level ISA [5]

Системные вызовы: откуда брать номера?

Например, read, write, mmap, munmap, exit, exit_group

```
$ /usr/bin/grep -e 'exit ' -e 'write ' -e 'openat ' \
    -e 'mmap ' -e 'munmap ' -e 'exit_group ' \
    /usr/riscv64-linux-gnu/include/asm-generic/unistd.h
#define __NR_openat 56
#define __NR_write 64
#define __NR_exit 93
#define __NR_exit_group 94
#define __NR_munmap 215
#define __NR3264_mmap 222
#define __NR_mmap __NR3264_mmap
```

N.B. Решение специфично для RISC-V и Ubuntu Linux

Системные вызовы: что подставлять вместо аргументов? (1/2)

- Изучать документацию, например: `man 2 mmap`
- Написать программу на Си и протрассировать

```c
#include <unistd.h>
#include <sys/syscall.h>
#include <stdio.h>

#define ARRAY_SIZE(arr) (sizeof(arr) / sizeof((arr)[0]))

int main(int argc, char *argv[]) {
  char buffer[12];
  FILE* fp = fopen("test.txt", "r");
  size_t ret = fread(buffer, sizeof(*buffer), ARRAY_SIZE(buffer), fp
  printf("%s", buffer);
  return 0;
}
```

Системные вызовы: что подставлять вместо аргументов? (2/2) I

```
$ export QEMU_STRACE=1
$ qemu-riscv64 -L /usr/riscv64-linux-gnu -cpu rv64 ./demo_in_c.
    exe
brk(NULL) = 0x0000555555559000
...
openat(AT_FDCWD,"/lib/libc.so.6",O_RDONLY|O_CLOEXEC) = 3
read(3,0xab2a8ec0,832) = 832
fstat(3,0x00002aaaab2a8d40) = 0
mmap(NULL,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,
    0) = 0x00002aaaab2f1000
mmap(NULL,1568176,PROT_EXEC|PROT_READ,MAP_PRIVATE|MAP_DENYWRITE
    ,3,0) = 0x00002aaaab2f3000
mmap(0x00002aaaab461000,20480,PROT_READ|PROT_WRITE,MAP_PRIVATE|
    MAP_DENYWRITE|MAP_FIXED,3,0x16e000) = 0x00002aaaab461000
```

Системные вызовы: что подставлять вместо аргументов? (2/2) II

```
mmap(0x00002aaaab466000,48560,PROT_READ|PROT_WRITE,MAP_PRIVATE|
   MAP_ANONYMOUS|MAP_FIXED,-1,0) = 0x00002aaaab466000
close(3) = 0
...
openat(AT_FDCWD,"test.txt",O_RDONLY) = 3
fstat(3,0x00002aaaab2a9a38) = 0
read(3,0x55559480,4096) = 19
fstat(1,0x00002aaaab2a9a38) = 0
write(1,0x5555a490,13)Some very lo
= 13
lseek(3,-7,SEEK_CUR) = 12
exit_group(0)
```

BONUS: Выделение памяти в куче (без libc)

https://godbolt.org/z/x1rWMPqaK

Порождение кода для переменных

Если все глобальные переменные известны, их можно хранить в стеке

```
addi sp, sp, -16
li t0, 1            # acc=1;
sd t0, (sp)
li t0, 6            # n=6;
sd t0, 8(sp)
...
ld t3, 8(sp)        # n:=n-1;
li t4, 1
sub t0, t3, t4
sd t0, 8(sp)
```

- Просто реализовать
- Неэффективно

Порождение кода для `while`

```
lab1:               # while
  ld t0, 8(sp)
  li t1, 1
  bge t1, t0, lab2 # (n>1) do
  ld t3, (sp)
  ld t4, 8(sp)
  mulw t0, t3, t4
  sd t0, (sp)       # acc=acc*n;
  ld t3, 8(sp)
  li t4, 1
  sub t0, t3, t4
  sd t0, 8(sp)      # n=n-1;
  j lab1
lab2:               # done
```

N.B. Для надежности можно делать метки для каждого цикла уникальными, но это, в принципе, не обязательно (см. раздел Labels [6])

Упражнение: подумайте, есть ли баг[a]

[a] https://five-embeddev.com/riscv-user-isa-manual/Priv-v1.12/m.html

AST для ассемблера (2/2)

```
type reg = ...
type instr =
  | Addi of reg * reg * int
  | ...

let addi k r1 r2 n = k (Addi (r1, r2, n))

let emit ?(comm = "") instr =
  instr (fun i -> save i comm somewhere)

(* Usage *)
let comm = sprintf "alloc variables caled %s" args_repr in
emit addi sp sp (-8 * count) ~comm
```

- Потом ассемблер можно анализировать
- type safety

- CISC vs. RISC
- Размеры программ[1]
- Macro-OPeration fusion [7, 8]

`slli rd, rs1, 32` srli rd, rd, 32	`rd = rs1 & 0×ffffffff` Clear upper word
`slli rd, rs1, {1,2,3}` add rd, rd, rs2 ld rd, 0(rd)	`rd = array[offset]` Три инструкции превращаются в Load Effective Address[2]

[1] https://github.com/Kakadu/riscv_amd64_demos/tree/master/fac (проверено: 4 октября 2024 г.)

[2] https://www.felixcloutier.com/x86/lea

Ссылки & Acknowledgements I

- Julia Evans. *How to look at the stack with gdb*. url: https://jvns.ca/blog/2021/05/17/how-to-look-at-the-stack-in-gdb.
- Raymond Chen. *The history of calling conventions, part 1*. 2004. url: https://devblogs.microsoft.com/oldnewthing/20040102-00/?p=41213.
- *System V Calling Convention*. 2024. url: https://gitlab.com/x86-psABIs/x86-64-ABI/-/jobs/artifacts/master/raw/x86-64-ABI/abi.pdf?job=build.
- *RISC-V ABIs Specification*. 2022. url: https://github.com/riscv-non-isa/riscv-elf-psabi-doc/blob/v1.0/riscv-cc.adoc.
- *The RISC-V Instruction Set Manual Volume I: User-Level ISA (official)*. 2024. url: https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj7Omv8tFtkp/view.

Ссылки & Acknowledgements II



RISC-V Assembly Programmer's Manual. 2017. url: https://github.com/riscv-non-isa/riscv-asm-manual/blob/main/riscv-asm.md.



Macro-Operation Fusion (wikichip). 2024. url: https://en.wikichip.org/wiki/macro-operation_fusion.



Christopher Celio и др. «The Renewed Case for the Reduced Instruction Set Computer: Avoiding ISA Bloat with Macro-Op Fusion for RISC-V». В: *CoRR* (2016). url: http://arxiv.org/abs/1607.02318.



SiFive. *All Aboard, Part 3: Linker Relaxation in the RISC-V Toolchain*. 2017. url: https://www.sifive.com/blog/all-aboard-part-3-linker-relaxation-in-riscv-toolchain (дата обр. 31.10.2024).